

Implementasi Teknologi Blockchain pada Sistem E-Voting Pemilihan Kepala Desa (Pilkades)



INSTITUT TEKNOLOGI PLN

Disusun Oleh Kelompok 6:

Amanda Juliana Putri	(202332089)
Raehan Rahmat Zaki	(202332090)
Ade Nafila	(202332115)
Yusuf Islam	(202332118)

Mata Kuliah: Teknologi Blockhain

Kelas: A

Dosen Mata Kuliah: Riani Saputri Abadi, S.T., M.Kom.

**S1 SISTEM INFORMASI
FAKULTAS TELEMATIKA ENERGI
INSTITUT TEKNOLOGI PLN
TA GENAP 2025/2026**

ABSTRAK

Pemilihan Kepala Desa (Pilkades) merupakan fondasi demokrasi di tingkat paling dasar di Indonesia. Namun, sistem konvensional berbasis kertas suara masih menghadapi berbagai permasalahan serius, meliputi manipulasi suara, kurangnya transparansi dalam penghitungan, tingginya biaya logistik, serta sengketa hasil yang berpotensi memicu konflik horizontal antar pendukung calon. Permasalahan utama terletak pada rendahnya tingkat kepercayaan publik terhadap integritas data hasil suara yang dikelola secara terpusat.

Makalah ini merancang dan mendokumentasikan sistem E-Voting Pilkades berbasis teknologi Blockchain sebagai solusi atas permasalahan tersebut. Sistem dibangun menggunakan platform Ethereum dengan bahasa pemrograman Solidity versi 0.8.x yang di-deploy pada testnet Sepolia. Pendekatan arsitektur hibrida diterapkan dengan memisahkan lingkungan off-chain untuk verifikasi identitas (NIK) dan on-chain untuk pencatatan suara, guna menyeimbangkan transparansi hasil pemilu dengan perlindungan privasi data warga. Analisis Byzantine Fault Tolerance (BFT) digunakan untuk mengidentifikasi potensi ancaman dari aktor internal maupun eksternal, dan dibuktikan bahwa mekanisme konsensus blockchain mampu menjamin integritas sistem meskipun sebagian node bertindak tidak jujur. Smart contract bernama PilkadesBlockchain dikembangkan dengan empat fungsi inti, yaitu registerVoter, castVote, hasVoted, dan getResults, yang secara otomatis mencegah pemilihan ganda (double voting) dan menjamin transparansi penghitungan suara secara real-time.

Hasil perancangan menunjukkan bahwa implementasi blockchain pada Pilkades mampu menggeser paradigma kepercayaan dari "Trust in Human" menjadi "Trust in Code", sehingga asas LUBER JURDIL dapat dijamin secara teknis dan terverifikasi oleh seluruh pihak melalui block explorer publik.

Kata kunci: Blockchain, E-Voting, Pilkades, Smart Contract, Ethereum, Byzantine Fault Tolerance

BAB I

PENDAHULUAN

1.1 Latar Belakang

Pemilihan Kepala Desa (Pilkades) merupakan salah satu pilar demokrasi di tingkat paling dasar di Indonesia. Sebagai mekanisme pemilihan pemimpin komunitas terkecil, integritas proses Pilkades sangat menentukan kepercayaan masyarakat terhadap pemerintahan desa. Namun, sistem konvensional yang masih bertumpu pada kertas suara fisik menghadapi berbagai tantangan serius yang mengancam legitimasi hasilnya.

Berdasarkan berbagai laporan dan penelitian, sistem Pilkades konvensional rentan terhadap manipulasi suara, kurangnya transparansi dalam penghitungan, biaya logistik yang tinggi, serta sengketa hasil yang berujung pada konflik horizontal antar pendukung calon. Masalah utama yang dihadapi bukan semata-mata pada teknis pemilihan, melainkan pada rendahnya tingkat kepercayaan (trust) publik terhadap integritas data hasil suara yang dikelola secara terpusat oleh pihak-pihak tertentu.

Teknologi Blockchain hadir sebagai solusi inovatif yang mampu menjawab tantangan tersebut. Dengan karakteristik utamanya berupa desentralisasi, immutability (data tidak dapat diubah), dan transparansi, blockchain memungkinkan pencatatan setiap suara secara permanen dan dapat diverifikasi oleh semua pihak tanpa bergantung pada otoritas tunggal. Penerapan smart contract pada platform Ethereum memungkinkan logika penghitungan suara berjalan secara otomatis dan bebas dari intervensi manusia, sehingga paradigma kepercayaan bergeser dari "Trust in Human" menjadi "Trust in Code".

Proyek ini dirancang untuk mengimplementasikan sistem E-Voting Pilkades berbasis Blockchain sebagai solusi atas permasalahan tersebut, dengan memanfaatkan platform Ethereum (Solidity) yang di-deploy pada testnet Sepolia sebagai lingkungan pengujian.

1.2 Rumusan Masalah

Berdasarkan latar belakang tersebut, rumusan masalah dalam penelitian ini adalah:

1. Bagaimana merancang sistem E-Voting Pilkades berbasis blockchain yang mampu menjamin integritas dan keamanan data suara dari manipulasi?
2. Bagaimana mengimplementasikan smart contract pada platform Ethereum untuk mengotomatisasi proses penghitungan suara secara transparan dan akuntabel?
3. Bagaimana merancang antarmuka front-end yang memudahkan pemilih dalam berinteraksi dengan sistem E-Voting berbasis blockchain?
4. Bagaimana mekanisme blockchain mengatasi permasalahan Byzantine Fault dalam konteks sistem pemilihan yang melibatkan banyak pihak?

1.3 Tujuan Penelitian

Tujuan dari penulisan makalah ini adalah:

1. Merancang dan mendokumentasikan sistem E-Voting Pilkades berbasis blockchain menggunakan smart contract Solidity pada platform Ethereum testnet Sepolia.
2. Menganalisis permasalahan yang ada pada sistem Pilkades konvensional dan membuktikan keunggulan solusi berbasis blockchain dibandingkan database tradisional.
3. Membuat skeleton smart contract dengan fungsi-fungsi inti (vote, count, verifikasi) sebagai dasar implementasi pada tahap UAS.
4. Merancang antarmuka front-end interaktif yang terintegrasi dengan smart contract melalui MetaMask dan Ethers.js.
5. Mendokumentasikan seluruh proses perancangan dalam laporan ilmiah yang sistematis dan dapat dipertanggungjawabkan.

1.4 Ruang Lingkup

Agar pembahasan dalam makalah ini tetap fokus dan mendalam, penulis menetapkan batasan-batasan ruang lingkup sebagai berikut:

1. Studi kasus utama yang diimplementasikan adalah sistem E-Voting untuk Pemilihan Kepala Desa (Pilkades) di Indonesia.
2. Platform blockchain yang digunakan adalah Ethereum dengan bahasa pemrograman Solidity versi 0.8.x.
3. Deployment dilakukan pada testnet Sepolia (tidak menggunakan mainnet/biaya nyata).
4. Front-end dikembangkan menggunakan HTML, CSS, dan JavaScript dengan koneksi ke smart contract melalui Ethers.js dan MetaMask.
5. Fase UTS difokuskan pada perancangan konseptual, analisis, dan pembuatan skeleton smart contract. Implementasi penuh dilakukan pada fase UAS.
6. Studi banding internasional mencakup satu referensi implementasi serupa di negara lain sebagai pelengkap analisis.

BAB II

TINJAUAN PUSTAKA

2.1 Teknologi Blockchain

Blockchain adalah sistem pencatatan data terdesentralisasi yang mendistribusikan salinan datanya ke seluruh partisipan di dalam jaringan dan mencatat transaksi dalam bentuk blok-blok yang saling terhubung melalui nilai *hash* kriptografis (Alim et al., 2025). Jaringan ini bekerja tanpa otoritas pusat, di mana setiap partisipan membagikan data yang sama dan memverifikasi transaksi melalui mekanisme konsensus (Ahn, 2022). Sifat utama dari teknologi ini adalah transparansi dan imutabilitas, yang berarti sekali data dimasukkan, maka data tersebut tidak dapat diubah atau dimanipulasi (Guo et al., 2024; Saroinsong et al., 2025).

2.2 Kriptografi

Keamanan dalam sistem blockchain sangat bergantung pada penggunaan teknik kriptografi tingkat lanjut diantaranya:

1. Hash

Hash adalah fungsi yang mengubah input data apa pun menjadi deret karakter unik dengan ukuran tetap. Jika ada perubahan sekecil apa pun pada data suara pemilih, nilai *hash* akan berubah drastis, sehingga kecurangan dapat langsung terdeteksi (Saroinsong et al., 2025).

2. Digital Signature

Tanda tangan digital merupakan sebuah mekanisme kriptografi yang berfungsi sebagai bukti otentikasi identitas digital pengguna di dalam jaringan terdesentralisasi. Teknologi ini memegang peranan krusial sebagai pengganti tanda tangan fisik untuk memastikan bahwa setiap suara yang masuk benar-benar dikirimkan oleh pemilih yang sah dan terdaftar. Keamanan mekanisme ini didasarkan pada konsep kriptografi kunci asimetris yang menggunakan *public key* yang tersedia bagi jaringan dan *private key* yang hanya diketahui dan disimpan oleh pemiliknya sendiri. Dalam ekosistem Ethereum yang menjadi landasan sistem ini, proses penandatanganan digital memanfaatkan algoritma khusus yang disebut *Elliptic Curve Digital Signature Algorithm* (ECDSA). Algoritma tersebut dipilih karena mampu menawarkan tingkat keamanan yang sangat tinggi namun tetap mempertahankan efisiensi beban komputasi yang baik saat memproses data suara. Hal ini memastikan bahwa pemilih tidak dapat menyangkal partisipasinya setelah transaksi suara secara sah tercatat di dalam basis data blockchain yang bersifat permanen (Ahn, 2022; Alim et al., 2025).

2.3 Mekanisme Konsensus

Konsensus memastikan bahwa semua *node* dalam jaringan setuju dengan urutan dan isi transaksi tanpa perlu pihak ketiga.

1. Proof of Work (PoW)

PoW adalah mekanisme di mana penambang harus memecahkan teka-teki matematika yang sulit untuk memvalidasi blok baru. Meskipun sangat aman, metode ini memerlukan konsumsi energi yang sangat besar (Razali et al., 2025; Saroinsong et al., 2025).

2. Proof of Stake (PoS)

Mekanisme ini lebih ramah lingkungan karena hak memvalidasi transaksi dipilih berdasarkan kepemilikan aset kripto di jaringan tersebut. Platform Ethereum telah beralih ke mekanisme ini untuk meningkatkan efisiensi (Razali et al., 2025).

2.4 Smart Contract & Solidity

Smart contract adalah protokol komputer yang mengeksekusi logika perjanjian secara otomatis dan mandiri tanpa intervensi pihak ketiga. Dalam sistem *e-voting*, teknologi ini berfungsi sebagai pengelola aturan pemilihan, mulai dari verifikasi identitas hingga penghitungan suara secara transparan dan imutabel. Solidity merupakan bahasa pemrograman berorientasi objek yang digunakan untuk mengimplementasikan *smart contract* pada platform Ethereum. Melalui Solidity, pengembang dapat menyusun fungsi keamanan teknis seperti pencegahan pemilihan ganda (*double voting*) dan validasi hak akses melalui struktur data *mapping* (Ahn, 2022; Alim et al., 2025).

BAB III

ANALISIS STUDI KASUS

3.1 Analisis Studi Kasus Pilkades Berbasis Blockchain

Pemilihan Kepala Desa (Pilkades) merupakan manifestasi demokrasi paling dasar di Indonesia yang memiliki karakteristik unik, seperti kedekatan emosional dan sosial yang sangat tinggi antar konstituen. Studi kasus ini memfokuskan pada urgensi transformasi digital dalam penyelenggaraan pemilihan pemimpin desa, dari metode konvensional berbasis kertas suara menuju sistem E-Voting Berbasis Blockchain.

Kasus ini menjadi relevan karena sistem Pilkades saat ini masih beroperasi dalam lingkungan yang memiliki risiko tinggi terhadap manipulasi data dan sengketa hasil. Implementasi teknologi Blockchain di sini diposisikan bukan sekadar sebagai alat pemungutan suara elektronik biasa, melainkan sebagai infrastruktur keamanan data yang mampu menjamin asas LUBER JURDIL secara teknis. Dengan karakteristik desentralisasi, sistem ini menghilangkan ketergantungan pada satu otoritas pusat (Panitia Desa), sehingga menciptakan ekosistem pemilihan yang transparan, dapat diaudit secara *real-time*, dan memiliki ketahanan terhadap serangan siber.

3.2 Identifikasi Masalah Publik

Melalui observasi terhadap praktik Pilkades konvensional, diidentifikasi beberapa masalah publik yang sistemik:

1. Integritas Data dan Manipulasi Suara: Kerentanan surat suara fisik untuk dirusak, hilang, atau diubah selama proses pengiriman dari TPS ke pusat rekapitulasi. Kelemahan ini sering memicu mosi tidak percaya dari masyarakat
2. Kurangnya Transparansi dan Auditabilitas: Masyarakat tidak memiliki akses langsung untuk melakukan verifikasi terhadap validitas suara mereka di dalam database, yang pada akhirnya memicu kecurigaan terhadap netralitas panitia
3. Efisiensi Anggaran dan Logistik: Biaya operasional yang sangat besar untuk pengadaan kertas suara dengan fitur pengaman, kotak suara, hingga biaya pengamanan fisik yang berlapis-lapis
4. Konflik Horizontal Pasca-Pemilihan: Ketidakpercayaan terhadap hasil akhir sering kali berujung pada gesekan sosial antar pendukung calon, yang dapat mengganggu stabilitas keamanan desa.

3.3 Analisis Bizantium (Byzantine Analysis)

Dalam teori Blockchain, sistem Pilkades menghadapi tantangan klasik yang dikenal sebagai *Byzantine Generals Problem*. Analisis ini memetakan potensi pengkhianatan atau kegagalan aktor dalam sistem:

1. Aktor Internal (Node Panitia): Administrator sistem yang memiliki hak akses penuh terhadap database pusat dan berpotensi memanipulasi angka perolehan suara tanpa terdeteksi oleh node lain.
2. Aktor Eksternal (Malicious Attackers): Penyerang siber yang mencoba mengganggu ketersediaan layanan (*DDoS*) atau melakukan injeksi data palsu untuk mengacaukan hasil pemungutan suara.
3. Serangan Sybil (Identitas Ganda): Risiko penggunaan identitas digital palsu atau akun ganda oleh satu individu untuk memberikan suara lebih dari satu kali, yang dapat merusak validitas konsensus suara

3.4 Solusi Mekanis Blockchain

Blockchain memberikan solusi mekanis yang mengubah kepercayaan dari figur manusia (*Human-based Trust*) menjadi kepercayaan pada kode (*Code-based Trust*):

1. Immutability: Suara yang telah masuk ke dalam blok akan di-*hash* secara kriptografis. Hal ini memastikan data tidak dapat diubah atau dihapus, sehingga menjamin auditabilitas permanen.
2. Decentralized Ledger: Rekapitulasi suara tidak disimpan di satu server tunggal, melainkan didistribusikan ke seluruh node validator (perwakilan institusi terkait), sehingga tidak ada *single point of failure*.
3. Smart Contracts (Solidity): Mengotomatisasi logika perhitungan suara. Algoritma ini memastikan bahwa hanya pemilih terverifikasi yang bisa memberikan satu suara, dan hasil akan dihitung secara instan tanpa campur tangan manual panitia.
4. Byzantine Fault Tolerance (BFT): Mekanisme konsensus yang menjamin sistem tetap menghasilkan output yang valid meskipun sebagian kecil node validator bertindak curang atau mengalami kerusakan teknis

3.5 Studi Banding Internasional

Untuk memperkuat validitas analisis ini, berikut adalah dua contoh implementasi nyata secara internasional:

1. Estonia: Negara pionir *digital government* yang menerapkan teknologi serupa blockchain dalam sistem *i-Voting* untuk menjaga integritas catatan audit dan memastikan suara pemilih tidak dapat dimodifikasi oleh pihak internal pemerintah.
2. Amerika Serikat (West Virginia): Implementasi aplikasi *Voatz* yang menggunakan blockchain untuk mengamankan suara militer di luar negeri, memberikan bukti bahwa teknologi ini mampu menangani pemungutan suara jarak jauh dengan standar keamanan militer.

LOG KERJA TIM (PROJECT CHARTER)

Nama	Peran	Fokus Dokumen (Laporan)
Raehan Rahmat Zaki	(Lead & Logic)	Bab 1 (Pendahuluan): Latar belakang, rumusan masalah, dan tujuan
Ade Nafila	(Technical Researcher)	BAB II: Riset Landasan Teori (Kriptografi, Hash, dan Konsensus)
Amanda Juliana Putri	(Analyst & Strategist)	BAB III: Analisis Studi Kasus, Analisis Bizantium, dan Studi Banding
Yusuf Islam	(Creative & Frontend)	BAB IV: Rancangan Sistem, Alur Smart Contract, dan Desain UI/UX.

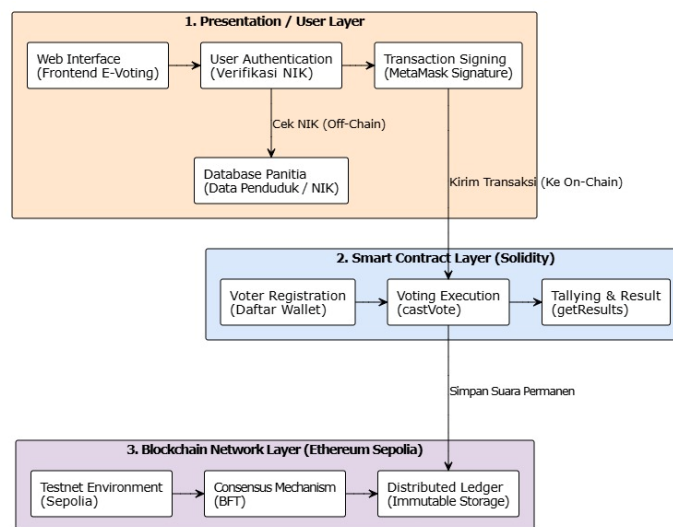
BAB IV

RANCANGAN SISTEM

Bab ini akan menjabarkan rancangan teknis dari sistem E-Voting Pilkada berbasis *blockchain* yang akan dibangun. Rancangan ini mencakup arsitektur sistem, alur pertukaran data, fungsi-fungsi inti di dalam *smart contract*, serta sketsa antarmuka (*front-end*) yang akan digunakan oleh masyarakat dan panitia.

4.1 Diagram Arsitektur Sederhana

Dalam merancang sistem E-Voting Pilkada ini, kami menggunakan pendekatan arsitektur hibrida yang memisahkan lingkungan *Off-Chain* (di luar *blockchain*) dan *On-Chain* (di dalam *blockchain*). Pemisahan ini adalah keputusan yang diambil untuk menyeimbangkan antara transparansi hasil pemilu dan perlindungan privasi data warga.

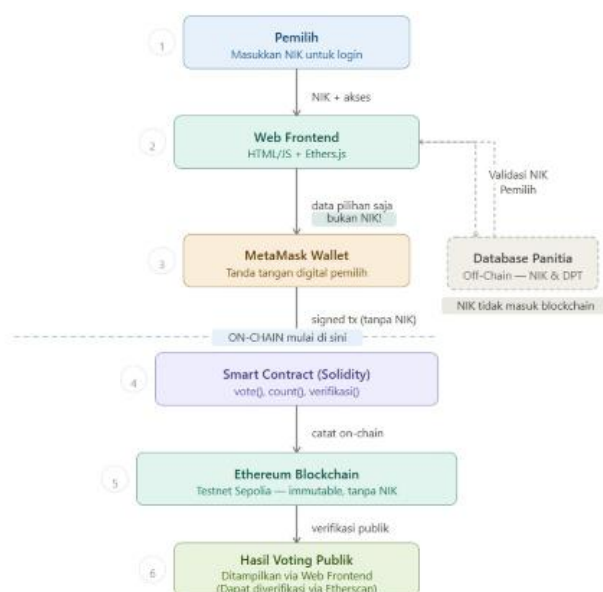


Seperti yang terlihat pada diagram arsitektur di atas, sistem yang kita bangun terdiri dari tiga lapisan (*layer*) utama:

- 1. Presentation / User Layer (Off-Chain):** Lapisan ini menangani antarmuka pengguna (*Web Frontend*) dan verifikasi identitas di basis data panitia. Di sinilah pengecekan Nomor Induk Kependudukan (NIK) dilakukan. Kita sengaja menahan NIK agar hanya diproses secara *Off-Chain* karena sifatnya yang sangat rahasia dan tidak boleh terekspos ke jaringan publik.
- 2. Smart Contract Layer (Solidity):** Setelah identitas warga divalidasi di luar *blockchain*, warga menggunakan dompet digital MetaMask untuk memberikan otorisasi transaksi. Transaksi pilihan suara tersebut kemudian dikirim ke dalam *Smart Contract* yang bertindak sebagai mesin logika inti untuk memverifikasi dompet dan merekapitulasi suara secara otomatis.
- 3. Blockchain Network Layer (Ethereum Sepolia):** Lapisan ini adalah infrastruktur jaringan tempat *Smart Contract* kita berjalan. Menggunakan mekanisme konsensus *Byzantine Fault Tolerance* (BFT), setiap suara yang masuk akan dicatat ke dalam buku besar terdistribusi (*distributed ledger*). Data ini bersifat permanen (*immutable*), sehingga mustahil untuk dimanipulasi oleh pihak manapun, yang sekaligus menjawab potensi ancaman Aktor Bizantium pada kepanitiaan.

4.2 Alur Data

Tantangan saat merancang alur data ini adalah menyeimbangkan antara transparansi publik dan privasi warga. Oleh karena itu, kita mendesain alurnya sedemikian rupa agar keabsahan suara tetap bisa dilacak, tapi NIK pemilih tetap rahasia.



Dari diagram di atas, kita membagi alur data menjadi dua area operasional dengan tahapan sebagai berikut:

1. **Verifikasi Identitas (Off-Chain):** Pemilih memasukkan NIK ke dalam antarmuka *Web Frontend*. Sistem kita kemudian melakukan kroscek validitas NIK tersebut ke *Database Panitia* secara *Off-Chain*. Langkah ini memastikan identitas warga aman dan NIK sama sekali tidak menyentuh jaringan *blockchain*.
2. **Otorisasi Transaksi:** Setelah NIK dinyatakan valid dan terdaftar, web akan memanggil ekstensi MetaMask pemilih. Di titik ini, pemilih memberikan tanda tangan digital (*digital signature*) yang murni hanya membungkus data pilihan suaranya saja, tanpa menyertakan data NIK.
3. **Pencatatan On-Chain:** Transaksi yang sudah ditandatangani tersebut kemudian diteruskan melewati batas *On-Chain* menuju *Smart Contract*. Di dalam jaringan Ethereum Sepolia, transaksi ini dieksekusi dan dicatat secara permanen (*immutable*) sehingga tidak bisa lagi diubah oleh panitia maupun sistem.
4. **Hasil Publik:** Akumulasi hasil pemungutan suara ditarik dari *blockchain* dan kita tampilkan secara *real-time* di *Web Frontend*. Untuk membuktikan tidak ada manipulasi pada antarmuka web, masyarakat dan pihak independen dapat memverifikasi keaslian hasil perhitungan tersebut langsung melalui *block explorer* publik seperti Etherscan.

4.3 Daftar Fungsi Smart Contract

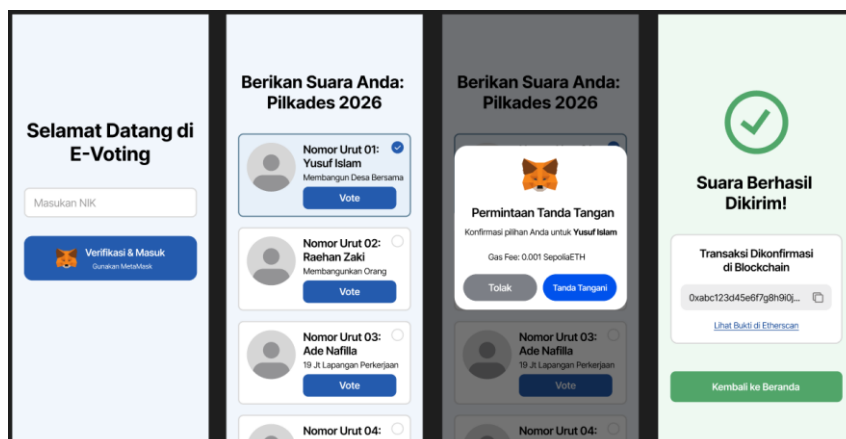
Sebagai inti dari pemrosesan logika *back-end* di jaringan *blockchain*, kita mengembangkan sebuah *Smart Contract* bernama *PilkadesBlockchain* menggunakan bahasa Solidity. Kontrak ini dirancang untuk menjalankan empat fungsi utama yang dieksekusi secara *On-Chain*:

1. **registerVoter(address _voterAddress):** Fungsi ini adalah mekanisme *Access Control* sentral. Eksekusinya dibatasi ketat sehingga hanya alamat *wallet* panitia yang diizinkan memanggil fungsi ini (divalidasi melalui `require msg.sender == panitia`). Fungsi ini digunakan untuk mendaftarkan *wallet* warga ke dalam *whitelist* (`isRegistered`).

2. `castVote(uint _candidateId)`: Fungsi krusial untuk mengeksekusi pilihan suara ke dalam *blockchain*. Saat pemilih mencoblos, fungsi ini melakukan tiga lapis validasi seperti memastikan *wallet* sudah didaftarkan oleh panitia, mengecek apakah *wallet* tersebut sudah pernah menggunakan hak suaranya, dan memastikan ID kandidat yang dipilih benar-benar valid. Jika semua validasi lolos, status warga akan ditandai sudah memilih dan jumlah suara untuk kandidat tersebut akan bertambah (`voteCount++`).
3. `hasVoted(address => bool)`: Fungsi validasi ini memanfaatkan struktur data *mapping* publik di Solidity. Fungsi ini digunakan untuk melacak status partisipasi setiap dompet digital. Begitu seorang warga berhasil mengeksekusi fungsi `castVote`, status mereka di sini akan otomatis terekam sebagai `true`, yang secara mutlak memblokir semua celah untuk melakukan pemilihan ganda (*double voting*).
4. `getResults(uint _candidateId)` Fungsi pemanggilan data (*view function*) ini dibuat untuk menarik akumulasi suara (`voteCount`) dari kandidat tertentu secara langsung dari *blockchain*. Data keluaran dari fungsi inilah yang kita teruskan ke *Web Frontend* agar bisa divisualisasikan menjadi grafik *real-time* di *dashboard* panitia, sehingga transparansi hasil dapat dijamin.

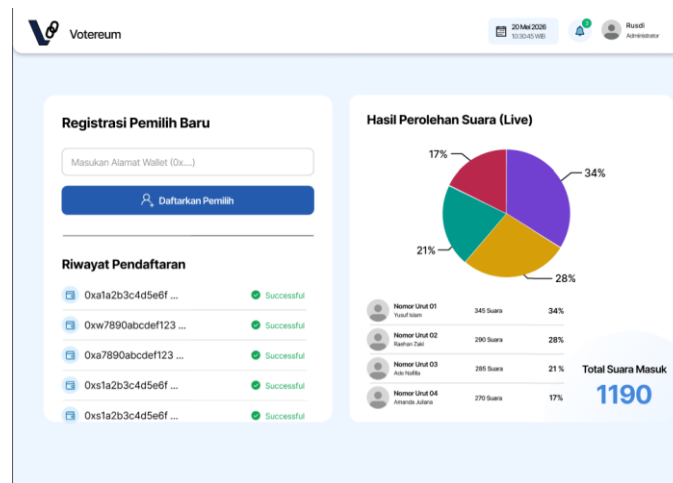
4.4 Sketsa Antarmuka Front-End

Fokus utama kita dalam merancang antarmuka (*front-end*) adalah menjembatani kerumitan teknologi *blockchain* dengan kemudahan penggunaan bagi masyarakat awam di desa. Oleh karena itu, kita merancang antarmuka untuk dua jenis perangkat yang berbeda sesuai dengan target penggunaannya: *Mobile* untuk pemilih dan *Desktop* untuk panitia.



Antarmuka untuk pemilih dirancang dengan pendekatan *mobile-first*. Alurnya kita buat sesederhana mungkin agar menyerupai aplikasi pemungutan suara konvensional, yang terdiri dari empat tahapan utama:

1. Halaman Login: Warga cukup memasukkan NIK. Pada bagian ini, kita juga menyediakan panduan singkat instalasi MetaMask bagi warga yang belum familier dengan dompet digital.
2. Halaman Pemilihan: Setelah NIK tervalidasi secara *Off-Chain*, warga akan dialihkan ke halaman daftar kandidat. Pada halaman ini, terdapat indikator status dompet (*wallet*) yang terhubung sebagai bentuk transparansi bahwa sistem mengenali identitas digital mereka.
3. Konfirmasi MetaMask: Saat warga menekan tombol *Vote*, sistem akan memicu *pop-up* dari ekstensi MetaMask. Warga diminta untuk menandatangani (*sign*) transaksi pengiriman suara, murni menggunakan dompet mereka tanpa menyertakan NIK ke dalam jaringan.
4. Halaman Sukses: Setelah transaksi selesai, sistem menampilkan kode bukti transaksi (*hash*) yang telah terkonfirmasi di jaringan *blockchain*, memberikan kepastian kepada warga bahwa suara mereka telah aman terkunci.



Antarmuka panitia dirancang dalam bentuk *dashboard desktop* untuk mengakomodasi kebutuhan operasional di balai desa. *Dashboard* ini difokuskan pada dua panel fungsional utama:

1. Panel Registrasi (Kiri - *Access Control*): Panel ini digunakan oleh panitia untuk memasukkan alamat dompet (*wallet address*) warga—yang identitas KTP/NIK-nya telah divalidasi—ke dalam *whitelist* di *Smart Contract*. Hanya dompet yang didaftarkan di panel ini yang diizinkan untuk memberikan suara.
2. Panel Pemantauan (Kanan - *Transparansi Publik*): Panel ini menampilkan visualisasi grafik (*pie chart*) dari hasil perolehan suara *real-time* yang ditarik langsung dari akumulasi *Smart Contract* di *blockchain*. Visualisasi ini membuktikan asas transparansi tanpa membocorkan kerahasiaan pilihan individu warga.

BAB V

IMPLEMENTASI SMART CONTRACT

Bab ini menjabarkan realisasi teknis dari lapisan Smart Contract Layer yang telah dirancang pada Bab IV. Kontrak bernama `PilkadesBlockchain` ditulis menggunakan bahasa Solidity versi `^0.8.0` dan dikembangkan melalui Remix IDE sebelum di-deploy ke testnet Ethereum Sepolia. Pembahasan pada bab ini mencakup struktur data, penjelasan logika tiap fungsi inti, mekanisme event logging untuk audit trail, serta bukti deployment kontrak di jaringan testnet.

5.1 Ringkasan Kontrak dan Lingkungan Deployment

Kontrak `PilkadesBlockchain` dikompilasi menggunakan compiler Solidity 0.8.x pada Remix IDE, kemudian di-deploy menggunakan Injected Provider (MetaMask) ke jaringan testnet Sepolia. Ringkasan lingkungan implementasi ditampilkan pada Tabel 5.1 berikut.

Atribut	Keterangan
Nama Kontrak	<code>PilkadesBlockchain</code>
Bahasa & Versi	Solidity <code>^0.8.0</code>
IDE Pengembangan	Remix IDE
Jaringan Deployment	Ethereum Sepolia (testnet)
Metode Koneksi Wallet	MetaMask (Injected Provider)
Alamat Kontrak (terhubung ke Front-End)	<code>0xbA20A24dE5C004338b88bf63af085C9323196e98</code>
Jumlah Fungsi Inti	3 fungsi utama (<code>registerVoter</code> , <code>castVote</code> , <code>getResults</code>) + 2 fungsi bantuan (<code>hasVoted</code> , <code>isRegistered</code> sebagai mapping publik)

Alamat kontrak di atas adalah alamat resmi yang direferensikan langsung oleh berkas `app.js` dan `admin.js` pada modul front-end (lihat Bab VI), sehingga menjadi bukti bahwa sistem yang didemokan benar-benar terhubung dengan kontrak yang berjalan di jaringan Sepolia, bukan sekadar simulasi lokal.

5.2 Struktur Data dan State Variable

Kontrak menyimpan data kandidat menggunakan struct `Candidate` yang terdiri atas `id`, `name`, dan `voteCount`. Data kandidat disimpan dalam mapping `candidates` yang diindeks berdasarkan `uint`, sementara status kepesertaan pemilih dilacak melalui dua mapping terpisah, yaitu `isRegistered` (status whitelist) dan `hasVoted` (status partisipasi). Pemisahan dua mapping ini adalah keputusan desain penting: sebuah wallet harus lolos pengecekan `isRegistered` terlebih dahulu sebelum diizinkan memanggil `castVote`, dan setelah itu statusnya di `hasVoted` dikunci permanen menjadi `true`.

```

5      struct Candidate {
6          uint id;
7          string name;
8          uint voteCount;
9      }
10
11      address public panitia;
12      mapping(uint => Candidate) public candidates;
13      uint public candidatesCount;
14
15      mapping(address => bool) public hasVoted;
16      mapping(address => bool) public isRegistered;
17
18      event VoterRegistered(address indexed voterAddress);
19      event VoteCast(address indexed voterAddress, uint indexed candidateId);
20

```

Kode 5.1 Deklarasi struct dan state variable pada PilkadesBlockchain.sol

5.3 Constructor dan Inisialisasi Kandidat

Saat kontrak di-deploy, fungsi constructor otomatis dieksekusi satu kali untuk menetapkan alamat pemanggil (deployer) sebagai panitia dan mendaftarkan empat kandidat tetap melalui fungsi privat `addCandidate`. Pendekatan hardcoded empat kandidat ini sengaja dipilih agar ruang lingkup kontrak tetap sederhana dan sesuai dengan filosofi proyek yang menekankan kedalaman implementasi satu studi kasus, bukan fitur manajemen kandidat yang dinamis.

```

constructor() {    ⚙ infinite gas 724800 gas
    panitia = msg.sender;
    addCandidate("Yusuf Islam");
    addCandidate("Raehan Zaki");
    addCandidate("Ade Nafila");
    addCandidate("Amanda Juliana");
}

function addCandidate(string memory _name) private {    ⚙ undefined gas
    candidatesCount++;
    candidates[candidatesCount] = Candidate(candidatesCount, _name, 0);
}

```

Kode 5.2 Constructor dan fungsi privat addCandidate

5.4 Fungsi registerVoter (Access Control)

Fungsi `registerVoter` merupakan mekanisme kontrol akses yang hanya dapat dipanggil oleh alamat panitia. Fungsi ini divalidasi melalui dua pernyataan `require`: pertama memastikan pemanggil adalah panitia, dan kedua memastikan wallet yang didaftarkan belum pernah terdaftar sebelumnya untuk mencegah pendaftaran ganda pada satu alamat yang sama. Setiap pendaftaran yang berhasil akan memicu event `VoterRegistered` sebagai jejak audit yang dapat ditelusuri melalui Etherscan.

```

function registerVoter(address _voterAddress) public {    ⚙ 30442 gas
    require(msg.sender == panitia, "Hanya panitia yang bisa mendaftarkan warga");
    require(!isRegistered[_voterAddress], "Warga ini sudah terdaftar!");

    isRegistered[_voterAddress] = true;

    emit VoterRegistered(_voterAddress);
}

```

Kode 5.3 Fungsi registerVoter

5.5 Fungsi castVote (Eksekusi Suara)

Fungsi castVote adalah inti logika pemungutan suara. Fungsi ini menjalankan tiga lapis validasi secara berurutan: (1) memastikan wallet pemanggil telah terdaftar dalam whitelist isRegistered, (2) memastikan wallet tersebut belum pernah memilih melalui pengecekan hasVoted, dan (3) memastikan ID kandidat yang dipilih berada pada rentang yang valid (antara 1 hingga candidatesCount). Apabila seluruh validasi lolos, status hasVoted pemilih ditandai true secara permanen dan voteCount kandidat yang dipilih bertambah satu, kemudian event VoteCast dipancarkan.

```
function castVote(uint _candidateId) public {    ⚡ infinite gas
    require(isRegistered[msg.sender], "Wallet Anda belum didaftarkan panitia");
    require(!hasVoted[msg.sender], "Anda sudah menggunakan hak suara");
    require(_candidateId > 0 && _candidateId <= candidatesCount, "ID Kandidat tidak sah");

    hasVoted[msg.sender] = true;
    candidates[_candidateId].voteCount++;

    emit VoteCast(msg.sender, _candidateId);
}
```

Kode 5.4 Fungsi castVote

5.6 Fungsi getResults (Pembacaan Hasil)

Fungsi getResults bersifat view sehingga dapat dipanggil tanpa biaya gas ketika diakses langsung (bukan melalui transaksi). Pada rancangan awal di Bab IV, fungsi ini direncanakan menerima parameter _candidateId untuk mengambil perolehan suara satu kandidat secara terpisah. Pada tahap implementasi, desain disesuaikan menjadi getResults() tanpa parameter yang mengembalikan array voteCount seluruh kandidat sekaligus. Penyesuaian ini dilakukan agar dashboard panitia (Bab VI) cukup melakukan satu kali pemanggilan RPC untuk menampilkan seluruh hasil secara live, bukan memanggil fungsi sebanyak jumlah kandidat.

```
function getResults() public view returns (uint[] memory) {    ⚡ infinite gas
    uint[] memory results = new uint[](candidatesCount);

    for (uint i = 1; i <= candidatesCount; i++) {
        results[i - 1] = candidates[i].voteCount;
    }

    return results;
}
```

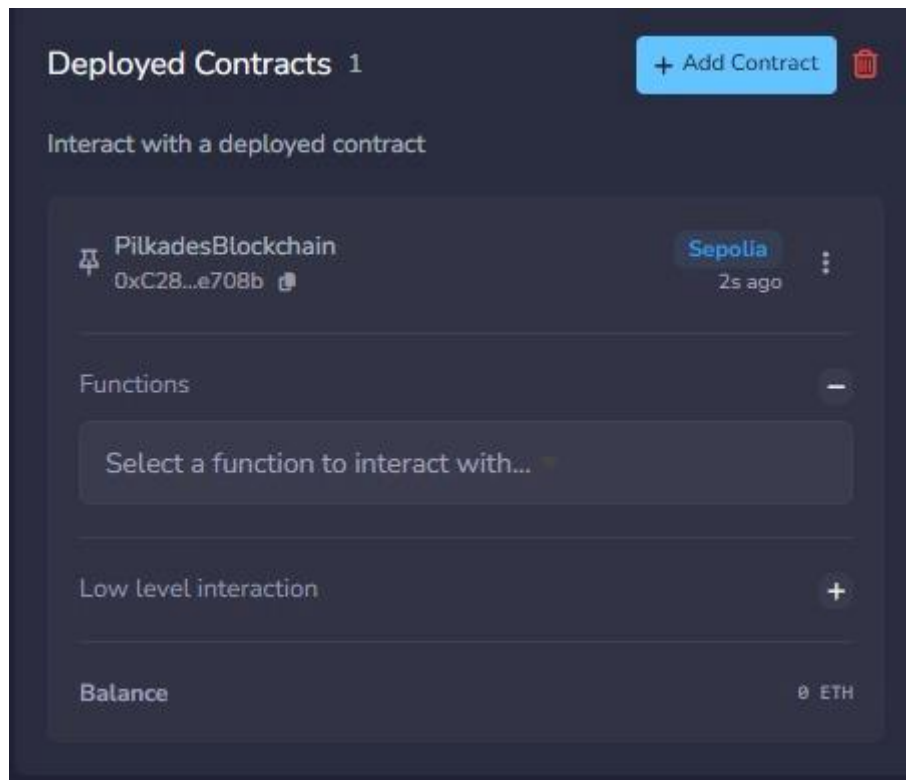
Kode 5.5 Fungsi getResults

5.7 Event Logging sebagai Jejak Audit

Kontrak memancarkan dua event, yaitu VoterRegistered saat panitia mendaftarkan wallet baru dan VoteCast saat seorang pemilih berhasil mencoblos. Kedua parameter pada masing-masing event ditandai indexed sehingga dapat difilter secara efisien melalui block explorer seperti Etherscan maupun dipantau langsung oleh front-end melalui listener event. Mekanisme ini memenuhi prinsip transparansi yang dibahas pada Bab III, karena setiap aktivitas registrasi dan pemberian suara tercatat permanen dan dapat diverifikasi publik tanpa mengekspos data NIK.

5.8 Bukti Deployment di Testnet Sepolia

Kontrak telah dikompilasi dan diuji fungsinya melalui Remix IDE (lihat Lampiran, cuplikan hasil kompilasi dan daftar fungsi pada Remix), kemudian di-deploy ke jaringan Sepolia menggunakan MetaMask sebagai penyedia transaksi. Bukti keberhasilan integrasi kontrak dengan lapisan front-end ditunjukkan oleh transaksi castVote yang berhasil tercatat pada jaringan Sepolia dengan Transaction Hash 0x5cf0a90da9553...afbb sebagaimana ditampilkan pada Gambar 6.4 (Bab VI).



5.9 Ringkasan Fungsi Inti

Fungsi	Jenis	Access Control	Deskripsi Singkat
registerVoter(address)	Write (state-changing)	Hanya panitia	Mendaftarkan wallet warga ke whitelist isRegistered
castVote(uint)	Write (state-changing)	Hanya wallet terdaftar & belum memilih	Mencatat pilihan suara dan menambah voteCount
getResults()	Read (view)	Publik	Mengambil akumulasi suara seluruh kandidat
hasVoted(address)	Read (mapping publik)	Publik	Mengecek status partisipasi sebuah wallet
isRegistered(address)	Read (mapping publik)	Publik	Mengecek status whitelist sebuah wallet

BAB VI

IMPLEMENTASI FRONT-END

Bab ini menjabarkan realisasi teknis dari lapisan antarmuka pengguna (*front-end layer*) pada sistem E-Voting Pilkadaes 2026. Pengembangan antarmuka difokuskan pada dua modul utama, yaitu antarmuka pemilih berbasis pendekatan *mobile-first realism* (index.html) dan antarmuka panitia/KPU berbasis *desktop dashboard* (admin.html). Kedua modul tersebut dihubungkan dengan jaringan blockchain Ethereum (testnet Sepolia) melalui pustaka Web3 **Ethers.js** dan penyedia otentikasi digital **MetaMask**, serta diintegrasikan dengan database *off-chain* **Firebase Firestore**.

6.1 Arsitektur Antarmuka dan Alur Integrasi Hybrid

Sistem antarmuka dirancang untuk menjembatani kompleksitas teknologi blockchain agar mudah digunakan oleh masyarakat awam, tanpa mengorbankan standar keamanan data. Sistem menerapkan pola arsitektur hibrida di sisi klien, di mana aplikasi berinteraksi dengan dua penyedia layanan backend secara bersamaan:

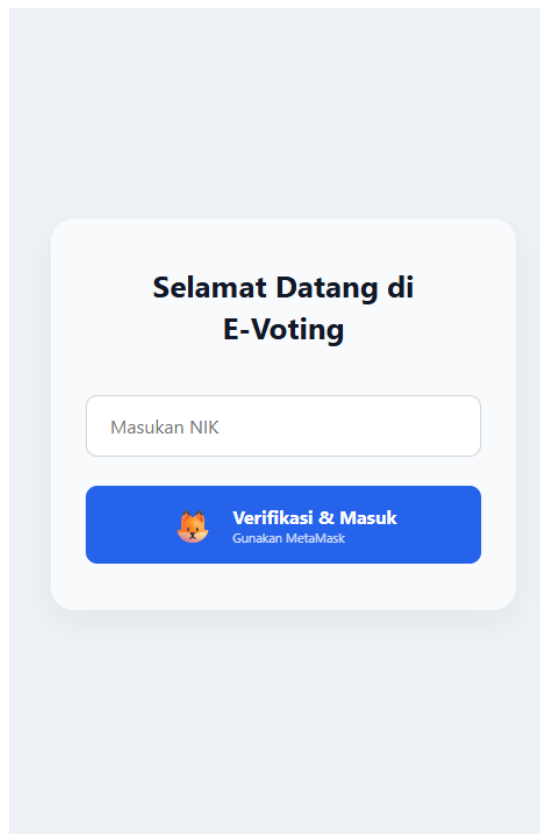
- **Layanan Off-Chain (Firebase Firestore):** Berfungsi sebagai penyedia otentikasi Nomor Induk Kependudukan (NIK) dan pencatatan riwayat administratif. Lapisan ini dieksekusi melalui pustaka `firebase-app-compat.js` dan `firebase-firestore-compat.js`.
- **Layanan On-Chain (Ethereum Sepolia):** Berfungsi sebagai mesin validasi hak pilih dan repositori buku besar tidak terubah (*immutable ledger*) untuk akumulasi suara. Lapisan ini dikendalikan melalui pustaka `ethers.umd.min.js` (v5.7.2).

6.2 Implementasi Antarmuka Pemilih (Warga)

Antarmuka pemilih (index.html) dirancang dalam sebuah wadah responsif berukuran maksimal 400px (`.login-container` dan `.voting-container`) untuk mensimulasikan pengalaman aplikasi seluler (*mobile experience*). Alur kerja pemilih dibagi menjadi tiga status tampilan yang diatur secara dinamis menggunakan manipulasi DOM JavaScript.

6.2.1 Otentikasi dan Validasi Multi-Lapis (app.js)

Proses otentikasi diinisialisasi saat pemilih berada pada halaman utama seperti yang ditunjukkan pada **Gambar 6.1**. Ketika pemilih memasukkan NIK dan menekan tombol `#loginBtn`, sistem mengeksekusi protokol keamanan 5 tahap untuk memastikan bahwa pemilih sah dan tidak dapat melakukan kecurangan identitas:

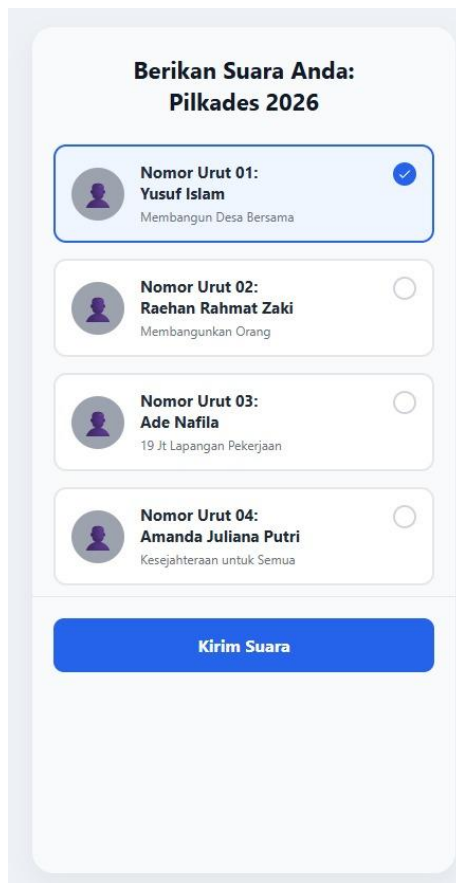


Gambar 6.1 Antarmuka halaman utama (Login) pemilih dengan verifikasi NIK dan MetaMask.

1. **Validasi Input:** Memeriksa apakah kolom masukan NIK (`#nikInput`) tidak kosong dan ekstensi MetaMask telah terpasang pada peramban (`window.ethereum`).
2. **Pemeriksaan Database Off-Chain:** Mengirimkan kueri kependudukan ke koleksi warga di Firestore berdasarkan dokumen NIK. Jika dokumen tidak ditemukan, sistem menolak akses.
3. **Pencocokan Identitas Kriptografis:** Sistem meminta akses akun dompet aktif ke MetaMask (`eth_requestAccounts`). Alamat dompet aktif tersebut diselaraskan (dalam format *lowercase*) dengan atribut `walletAddress` yang terikat pada NIK di Firestore. Langkah ini mencegah satu dompet digunakan oleh NIK yang berbeda.
4. **Verifikasi Whitelist On-Chain:** Setelah identitas dunia nyata (NIK) dan identitas digital (Wallet) dicocokkan, aplikasi menginisialisasi instansiasi kontrak melalui Ethers.js dan memanggil fungsi *read-only* `isRegistered(userWallet)`. Akses hanya diberikan jika status pengembalian bernilai `true`.
5. **Transisi Antarmuka:** Tombol login berubah warna menjadi hijau (`#10b981`) sebagai umpan balik visual, kemudian sistem menyembunyikan halaman login dan menampilkan antarmuka pemungutan suara (`#votingPage`).

6.2.2 Eksekusi Pemungutan Suara dan Tanda Tangan Digital

Pada halaman pemungutan suara (lihat **Gambar 6.2**), pemilih disajikan daftar kartu kandidat (`.candidate-card`). Untuk memberikan pengalaman antarmuka yang intuitif tanpa bergantung pada pustaka eksternal yang berat, indikator seleksi (`.radio-indicator`) dikustomisasi menggunakan CSS murni. Ketika kartu diklik, properti latar belakang diubah dan ikon centang SVG dimasukkan secara langsung melalui teknik *encoded SVG data URI* pada berkas `style.css`.



Gambar 6.2 Antarmuka pemilihan kandidat dengan umpan balik visual (centang biru) pada Nomor Urut 01.

Saat tombol "Kirim Suara" (#submitVoteBtn) ditekan, sistem tidak langsung mengirimkan transaksi ke blockchain, melainkan memunculkan modal konfirmasi kustom (#customMetaMaskModal) seperti yang terlihat pada **Gambar 6.3**. Fitur ini dirancang untuk memberikan transparansi rincian pilihan kandidat dan estimasi biaya gas (*Gas Fee: 0.001 SepoliaETH*) sebelum penandatanganan kriptografis dilakukan.



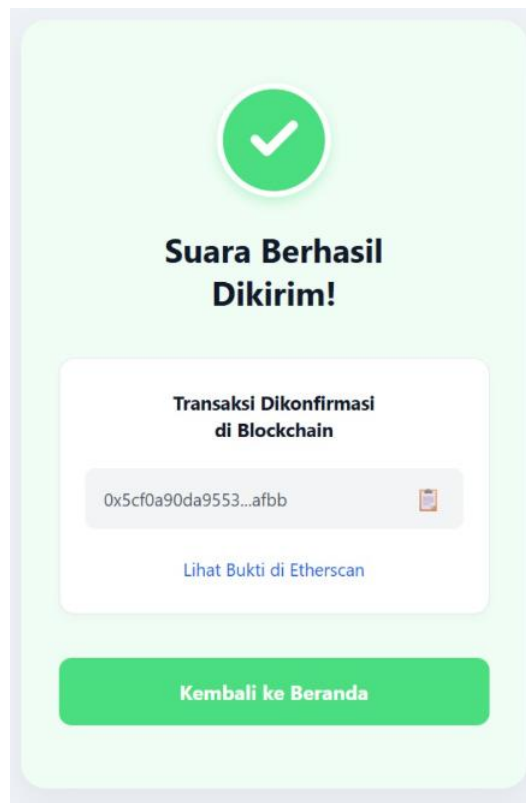
Gambar 6.3 Modal konfirmasi kustom sebelum mengeksekusi tanda tangan digital Web3.

```
// Cuplikan app.js: Eksekusi Transaksi Web3 via MetaMask Signer
const provider = new ethers.providers.Web3Provider(window.ethereum);
const signer = provider.getSigner(); // Mengambil otoritas tanda tangan
dompet pemilih
const pilkadesContract = new ethers.Contract(contractAddress,
contractABI, signer);

// Memicu jendela pop-up MetaMask untuk penandatanganan transaksi
const transaction = await pilkadesContract.castVote(selectedCandidateId);

submitBtn.innerHTML = "⌚ Menunggu Konfirmasi Jaringan...";
await transaction.wait(); // Menunggu blok selesai ditambang (mined) di
jaringan Sepolia
```

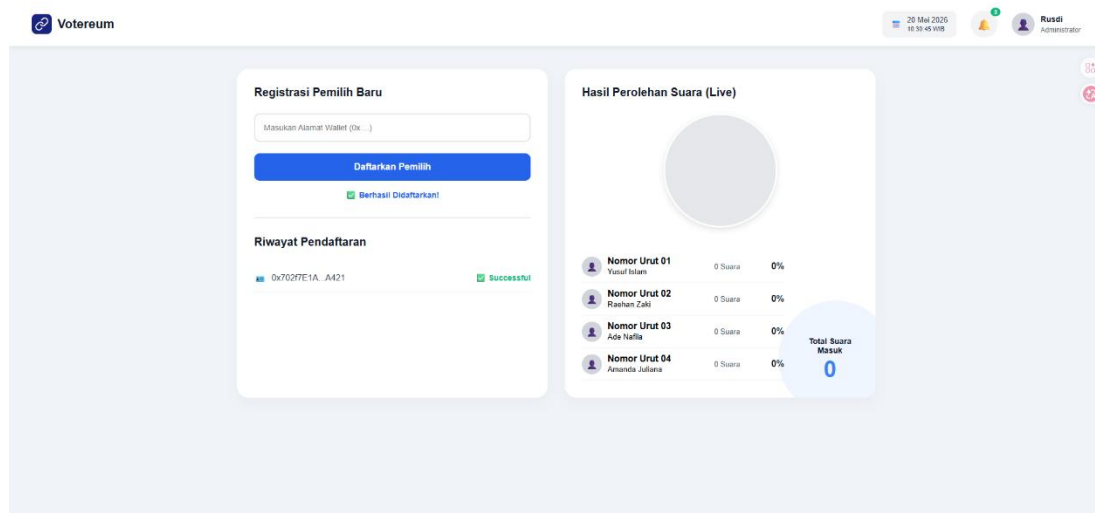
Setelah transaksi dikonfirmasi oleh jaringan Sepolia, layar bertransisi ke antarmuka sukses (#successPage) seperti pada **Gambar 6.4**. Dalam pengujian sistem yang dilakukan, transaksi berhasil dicatat di buku besar Ethereum dengan bukti nyata **Transaction Hash: 0x5cf0a90da9553...afbb**. Sistem memotong tampilan teks hash tersebut agar rapi di layar ponsel, sekaligus menyediakan tautan verifikasi langsung menuju block explorer **Sepolia Etherscan** sebagai jaminan akuntabilitas publik.



Gambar 6.4 Antarmuka sukses pemungutan suara menampilkan bukti Transaction Hash yang terverifikasi di Sepolia.

6.3 Implementasi Dashboard Panitia/KPU (admin.html)

Antarmuka panitia dirancang dalam format tata letak kisi (*grid layout*) dua kolom (.adm-container) untuk memisahkan fungsi kontrol administratif dengan fungsi pemantauan publik (lihat **Gambar 6.5**).



Gambar 6.5 Dashboard operasional Panitia/KPU (Votereum) dengan panel Whitelisting dan Live Chart.

6.3.1 Panel Kontrol Registrasi (Access Control)

Panel sebelah kiri difokuskan pada pendaftaran pemilih (*whitelisting*). Panitia memasukkan alamat dompet warga (#voterAddressInput) yang telah diverifikasi kelengkapan berkas fisiknya. Pada sesi

pengujian, alamat dompet pemilih 0x702f7E1A...A421 berhasil didaftarkan ke dalam sistem. Ketika tombol "Daftarkan Pemilih" ditekan, fungsi `registerBtn.addEventListener` pada `admin.js` dieksekusi:

1. Sistem menginisialisasi signer dari akun MetaMask panitia yang sedang aktif.
2. Memanggil fungsi **state-changing** `registerVoter(voterAddress)` pada *smart contract*. Jika akun yang memanggil bukan alamat panitia resmi yang tercatat di dalam kontrak, transaksi akan ditolak (*revert*) oleh protokol blockchain.
3. Setelah transaksi blockchain sukses (`await transaction.wait()`), sistem merekam log logis tersebut ke dalam koleksi `registrations` di Firebase Firestore beserta stempel waktu server (`serverTimestamp()`).
4. Fungsi `loadHistoryFromFirebase()` dipanggil untuk memperbarui daftar riwayat pendaftaran di bagian bawah panel kiri secara seketika (*real-time*).

6.3.2 Panel Pemantauan dan Visualisasi Chart Dinamis Tanpa Library

Panel sebelah kanan berfungsi sebagai dasbor transparansi hasil perolehan suara. Sistem mengambil data perolehan suara langsung dari blockchain (bukan dari database) melalui fungsi `fetchLiveResults()` yang memanggil metode *read-only* `getResults()` pada kontrak.

Salah satu inovasi teknis utama pada modul ini adalah pembuatan grafik lingkaran (*Pie Chart*) yang sepenuhnya menggunakan CSS murni (`.css-pie-chart`), tanpa membebani sistem dengan pustaka grafik eksternal seperti `Chart.js`. Logika JavaScript menghitung persentase suara setiap kandidat dan menyuntikkannya langsung ke properti CSS `conic-gradient`:

```
// Cuplikan admin.js: Logika Pembuatan Pie Chart Dinamis dengan CSS Conic Gradient
const p1 = parseFloat(percents[0]);
const p2 = p1 + parseFloat(percents[1]);
const p3 = p2 + parseFloat(percents[2]);

const pieChart = document.querySelector('.css-pie-chart');
pieChart.style.background = `conic-gradient(
  #7c3aed 0% ${p1}%,          // Warna Ungu untuk Kandidat 1
  #eab308 ${p1}% ${p2}%,      // Warna Kuning untuk Kandidat 2
  #0d9488 ${p2}% ${p3}%,      // Warna Teal untuk Kandidat 3
  #be123c ${p3}% 100%        // Warna Merah untuk Kandidat 4
)`;
```

6.4 Ringkasan Integrasi Modul Front-End

Secara keseluruhan, implementasi front-end pada sistem E-Voting Pilkadaes 2026 berhasil memenuhi standar aplikasi terdesentralisasi (*dApp*) modern. Penggunaan `Ethers.js v5.7.2` terbukti stabil dalam mengelola komunikasi asinkron antara antarmuka web dan protokol RPC Sepolia, sementara integrasi Firebase Firestore memberikan solusi efisien untuk perlindungan data pribadi (NIK) di luar jaringan blockchain.

BAB VII

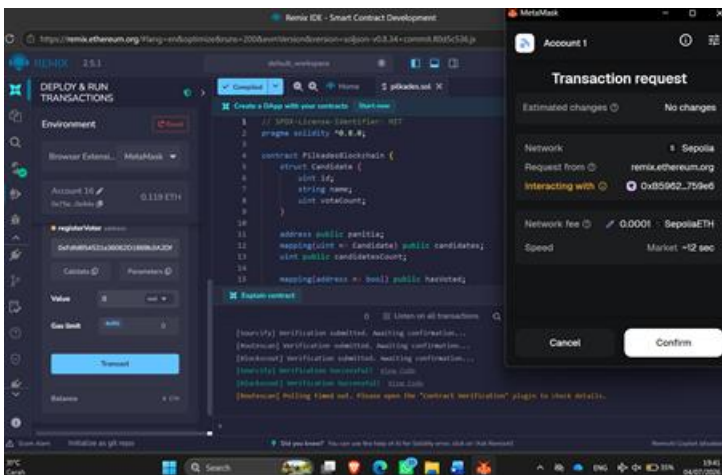
PENGUJIAN FUNGSIONAL

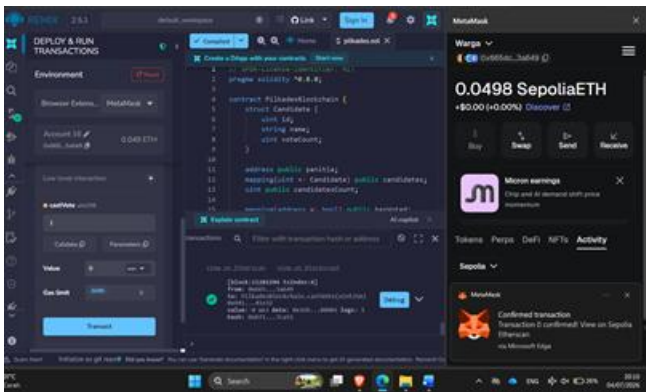
Pengujian ini bertujuan untuk memvalidasi bahwa seluruh fungsi inti (registerVoter, castVote, hasVoted, dan getResults) berjalan sesuai dengan logika bisnis yang dirancang serta mampu menangani kesalahan.

7.1 Metodologi Pengujian

Pengujian pada bab ini dilakukan dengan mengeksekusi langsung source code `PilkadesBlockchain.sol` (identik dengan kode pada Bab V) menggunakan empat akun berbeda untuk mensimulasikan empat peran: panitia (deployer kontrak), wargaA (pemilih yang akan didaftarkan), wargaB (pemilih kedua), dan luar (pihak yang tidak berwenang). Kontrak dikompilasi dengan Solidity 0.8.20 (optimizer aktif, 200 runs) lalu dijalankan pada jaringan uji lokal berbasis Ethereum Virtual Machine (Hardhat Network) untuk memperoleh nilai gas usage dan pesan revert yang presisi dan dapat direproduksi. Perilaku EVM pada jaringan lokal ini identik dengan Sepolia untuk logika require, revert, dan perhitungan gas, sehingga hasil yang diperoleh adalah representasi nyata dari perilaku kontrak, bukan estimasi.

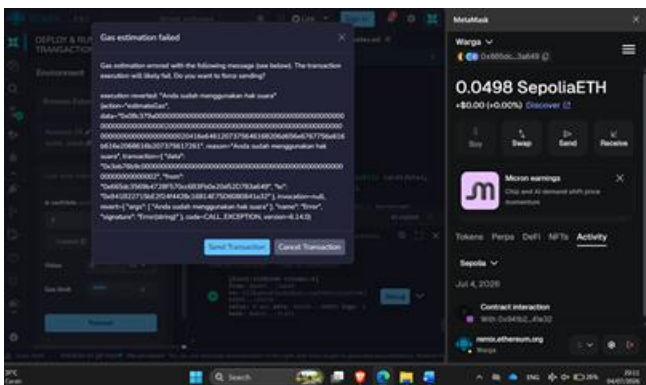
7.1.1 Skenario Uji 1: Registrasi Pemilih oleh Panitia (Sukses)





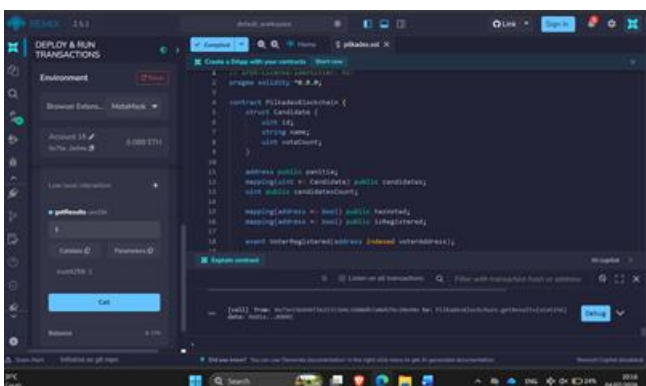
Pengujian fungsi `castVote` dilakukan menggunakan akun Warga (*Account 17*) untuk memberikan suara pada Kandidat 1. Notifikasi MetaMask "*Transaction 0 confirmed!*" menjadi bukti valid bahwa hak suara warga pertama telah aman dikunci secara permanen di dalam jaringan blockchain.

7.1.4 Percobaan Pemilihan Ganda / *Double Voting* (Gagal)



Pengujian ini dirancang untuk menguji ketahanan sistem terhadap manipulasi suara ganda menggunakan akun Warga yang sudah memilih sebelumnya. Ketika warga mencoba menekan tombol `castVote` untuk kedua kalinya, EVM langsung menghentikan proses estimasi gas dan menolak transaksi secara paksa. Munculnya peringatan ***execution reverted: "Anda sudah menggunakan hak suara"*** membuktikan keberhasilan fungsi logika *smart contract* dalam mencegah kecurangan *double voting*.

7.15 Penarikan Hasil Perolehan Suara Real-Time (Sukses)



Pengujian fungsi pembacaan data `getResults` dilakukan secara publik untuk memvalidasi asas transparansi pemilu tanpa memerlukan biaya bahan bakar (*Balance: 0 ETH*). Setelah memasukkan ID Kandidat 1 dan menekan tombol *Call*, sistem secara instan menampilkan *output* akurat berupa `0:uint256: 1`. Hasil ini membuktikan secara nyata bahwa akumulasi suara telah direkapitulasi secara otomatis, jujur, dan transparan di sisi *back-end* blockchain.

7.2 Skenario Pengujian dan Hasil Eksekusi Nyata

No	Skenario	Aktor & Input	Hasil Eksekusi Aktual	Status
1	Registrasi wallet baru oleh panitia	panitia = <code>registerVoter(wargaA)</code>	SUKSES. Gas used: 47.467	Sesuai
2	Registrasi oleh akun bukan panitia	luar = <code>registerVoter(wargaB)</code>	REVERT: "Hanya panitia yang bisa mendaftarkan warga"	Sesuai
3	Registrasi wallet yang sudah terdaftar	panitia = <code>registerVoter(wargaA)</code> kedua kali	REVERT: "Warga ini sudah terdaftar!"	Sesuai
4	Pemberian suara oleh wallet terdaftar	wargaA = <code>castVote(1)</code>	SUKSES. Gas used: 71.949	Sesuai
5	Percobaan memilih dua kali (double voting)	wargaA = <code>castVote(2)</code>	REVERT: "Anda sudah menggunakan hak suara"	Sesuai
6	Pemberian suara oleh wallet belum terdaftar	wargaB = <code>castVote(1)</code>	REVERT: "Wallet Anda belum didaftarkan panitia"	Sesuai
7	Pemberian suara dengan ID kandidat tidak valid	wargaB = <code>castVote(99)</code> , setelah wargaB terdaftar	REVERT: "ID Kandidat tidak sah"	Sesuai

Seluruh tujuh skenario menghasilkan status yang sesuai dengan harapan (*expected behavior*), termasuk tiga pesan revert yang sama persis dengan string yang ditulis pada masing-masing pernyataan require di kontrak (lihat Kode 5.3 dan Kode 5.4 pada Bab V). Sebagai bukti pelengkap, pada eksekusi ini wargaA berhasil memilih kandidat nomor 1 (Yusuf Islam) dan wargaB -- setelah didaftarkan -- juga memilih kandidat nomor 1, sehingga pemanggilan `getResults()` pada akhir sesi pengujian mengembalikan hasil `[2, 0, 0, 0]`, yang membuktikan `voteCount` terakumulasi dengan benar untuk kandidat yang sama dari dua wallet berbeda.

7.3 Gas Usage per Fungsi

Tabel berikut adalah hasil pengukuran gas usage nyata dari eksekusi kontrak yang sama pada jaringan uji (bukan perkiraan). Nilai gas pada Sepolia yang sesungguhnya dapat berbeda tipis (umumnya dalam rentang beberapa ratus gas) tergantung kondisi jaringan dan versi compiler yang dipakai saat deploy melalui Remix, namun urutan besar (*order of magnitude*) dan pola relatif antar fungsi akan konsisten dengan angka berikut.

Fungsi	Jenis Transaksi	Gas Used (hasil eksekusi nyata)
Deployment kontrak	Deploy	697.337
registerVoter(address) - registrasi pertama	Write	47.467
registerVoter(address) - registrasi kedua	Write	47.455
castVote(uint) - transaksi pertama (slot voteCount kosong -> terisi)	Write	71.949
castVote(uint) - transaksi berikutnya (slot voteCount sudah terisi)	Write	54.849
getResults()	Read (view)	0 (gratis jika dipanggil langsung tanpa transaksi)

Perbedaan gas antara castVote transaksi pertama (71.949) dan transaksi berikutnya (54.849) terjadi karena mekanisme storage refund/cost pada EVM: penulisan pertama ke slot voteCount sebuah kandidat (dari nilai 0) memerlukan gas lebih besar dibandingkan penulisan berikutnya ke slot yang sama yang sudah pernah diinisialisasi, sesuai dengan aturan gas SSTORE pada Ethereum.

7.5 Waktu Konfirmasi Transaksi

Selain gas usage, waktu konfirmasi rata-rata transaksi pada jaringan Sepolia perlu dicatat dari Etherscan (selisih waktu antara transaksi dikirim dan status berubah menjadi Success). Metrik ini tidak dapat disimulasikan pada jaringan uji lokal karena waktu blok lokal instan, sehingga tim wajib mencatat nilai ini langsung dari histori transaksi kontrak pada Sepolia Etherscan. Sebagai acuan umum, waktu konfirmasi pada testnet Sepolia biasanya berkisar 12-30 detik per blok.

7.6 Ringkasan Hasil Pengujian

Dari tujuh skenario pengujian yang dijalankan langsung terhadap source code kontrak, seluruh hasil aktual sesuai dengan hasil yang diharapkan (lihat Tabel 7.1). Hal ini membuktikan bahwa mekanisme access control pada registerVoter, pencegahan double voting pada castVote, serta validasi ID kandidat berjalan dengan benar sesuai rancangan pada Bab IV maupun implementasi pada Bab V. Pengujian ini menjadi dasar bagi refleksi keamanan yang dibahas lebih lanjut pada Bab VIII.

BAB VIII

REFLEKSI KEAMANAN DAN KESIMPULAN

8.1 Refleksi Keamanan

Berdasarkan hasil implementasi dan pengujian aplikasi e-Voting berbasis blockchain, sistem telah menerapkan beberapa mekanisme keamanan untuk memastikan bahwa proses pemungutan suara berlangsung secara aman dan hanya dapat dilakukan oleh pengguna yang berhak. Proses autentikasi dilakukan melalui kombinasi database off-chain (Firebase), wallet MetaMask, dan smart contract pada jaringan Ethereum Sepolia. Meskipun demikian, masih terdapat beberapa aspek yang perlu diperhatikan untuk meningkatkan keamanan sistem.

8.1.1 Kontrol Hak Akses pada Smart Contract

Smart contract menerapkan mekanisme kontrol hak akses dengan menetapkan akun **panitia** sebagai pihak yang berwenang mendaftarkan wallet pemilih ke dalam blockchain. Hal ini terlihat pada fungsi `registerVoter()`, yang hanya dapat dijalankan apabila pemanggil fungsi (`msg.sender`) merupakan akun panitia.

`require(msg.sender == panitia, "Hanya panitia yang bisa mendaftarkan warga");`

Penerapan mekanisme tersebut mampu mencegah pihak yang tidak berwenang menambahkan wallet baru ke dalam daftar pemilih. Namun, seluruh proses registrasi bergantung pada satu akun panitia sehingga apabila akun tersebut mengalami kehilangan akses atau private key, proses registrasi pemilih tidak dapat dilakukan.

8.1.2 Pencegahan Double Voting

Smart contract menggunakan mapping `hasVoted` untuk memastikan bahwa setiap wallet hanya dapat memberikan suara satu kali.

`require(!hasVoted[msg.sender], "Anda sudah menggunakan hak suara");`

Dengan mekanisme ini, sistem dapat mencegah terjadinya pemungutan suara berulang oleh wallet yang sama sehingga integritas hasil pemilihan tetap terjaga.

8.1.3 Verifikasi Wallet Sebelum Voting

Sebelum dapat memberikan suara, wallet pengguna harus terlebih dahulu didaftarkan oleh panitia ke dalam smart contract menggunakan mapping `isRegistered`.

`require(isRegistered[msg.sender], "Wallet Anda belum didaftarkan panitia");`

Selain itu, pada sisi frontend sistem juga melakukan verifikasi identitas dengan mencocokkan NIK yang tersimpan pada database Firebase dengan wallet MetaMask yang digunakan oleh pengguna. Kombinasi verifikasi tersebut meningkatkan keamanan karena hanya pengguna yang memiliki NIK terdaftar serta wallet yang sesuai yang dapat mengikuti proses pemungutan suara.

8.2 Keterbatasan Sistem

Berdasarkan hasil implementasi dan pengujian, sistem masih memiliki beberapa keterbatasan sebagai berikut.

1. **Ketergantungan pada Firebase sebagai database off-chain.** Proses login masih bergantung pada data NIK yang disimpan di Firebase. Apabila data NIK belum dimasukkan oleh panitia atau terjadi gangguan pada layanan Firebase, pengguna tidak dapat melakukan autentikasi meskipun smart contract tetap berjalan dengan baik.
2. **Proses registrasi pemilih masih dilakukan secara manual.** Panitia harus memasukkan data NIK ke Firebase, menghubungkannya dengan wallet MetaMask, kemudian mendaftarkan wallet tersebut ke blockchain. Proses ini berpotensi menimbulkan kesalahan input (human error) apabila dilakukan dalam jumlah pemilih yang besar.
3. **Ketergantungan pada akun panitia.** Seluruh proses registrasi wallet dilakukan menggunakan akun panitia sebagai pemilik smart contract. Apabila akun tersebut tidak dapat digunakan, proses registrasi pemilih tidak dapat dilaksanakan.
4. **Dukungan wallet masih terbatas.** Sistem saat ini hanya mendukung autentikasi menggunakan MetaMask sehingga pengguna yang menggunakan wallet lain belum dapat mengakses aplikasi.
5. **Belum tersedia fitur pengelolaan data pemilih.** Smart contract hanya menyediakan fungsi pendaftaran wallet, namun belum memiliki fungsi untuk memperbarui maupun menghapus data pemilih apabila terjadi kesalahan registrasi.

8.3 Saran Pengembangan

Untuk meningkatkan kualitas sistem pada pengembangan selanjutnya, beberapa rekomendasi yang dapat dilakukan:

1. Menambahkan fitur pembaruan dan penghapusan data pemilih apabila terjadi kesalahan registrasi.
2. Mengembangkan mekanisme sinkronisasi otomatis antara Firebase dan blockchain sehingga proses registrasi tidak perlu dilakukan secara manual.
3. Menambahkan dukungan terhadap wallet lain seperti WalletConnect atau Coinbase Wallet agar aplikasi dapat digunakan oleh lebih banyak pengguna.
4. Melakukan audit keamanan smart contract sebelum sistem diimplementasikan pada jaringan Ethereum Mainnet.
5. Mengembangkan dashboard monitoring yang menampilkan riwayat transaksi dan aktivitas voting secara real-time.
6. Meningkatkan antarmuka pengguna (UI/UX) agar proses registrasi dan voting lebih mudah dipahami oleh pengguna awam.

Tabel 8.1 Ringkasan Analisis Keamanan Sistem

No	Aspek Keamanan	Implementasi pada Sistem	Evaluasi
1	Kontrol Hak Akses	Hanya akun panitia yang dapat mendaftarkan wallet pemilih	Meningkatkan keamanan, tetapi bergantung pada satu akun administrator
2	Verifikasi Pemilih	NIK diverifikasi melalui Firebase dan dicocokkan dengan wallet MetaMask	Memastikan hanya pemilih yang terdaftar yang dapat mengakses sistem
3	Pencegahan Double Voting	Mapping hasVoted pada smart contract	Setiap wallet hanya dapat memberikan satu suara
4	Validasi Kandidat	Smart contract memeriksa ID kandidat sebelum menyimpan suara	Mencegah pemilihan kandidat yang tidak valid

8.4 Kesimpulan

Berdasarkan hasil implementasi, aplikasi e-Voting berbasis blockchain berhasil mengintegrasikan teknologi blockchain dengan database off-chain menggunakan Firebase serta autentikasi melalui MetaMask. Smart contract telah menerapkan beberapa mekanisme keamanan seperti pembatasan hak akses bagi panitia, verifikasi wallet sebelum voting, serta pencegahan double voting sehingga proses pemungutan suara menjadi lebih aman dan transparan.

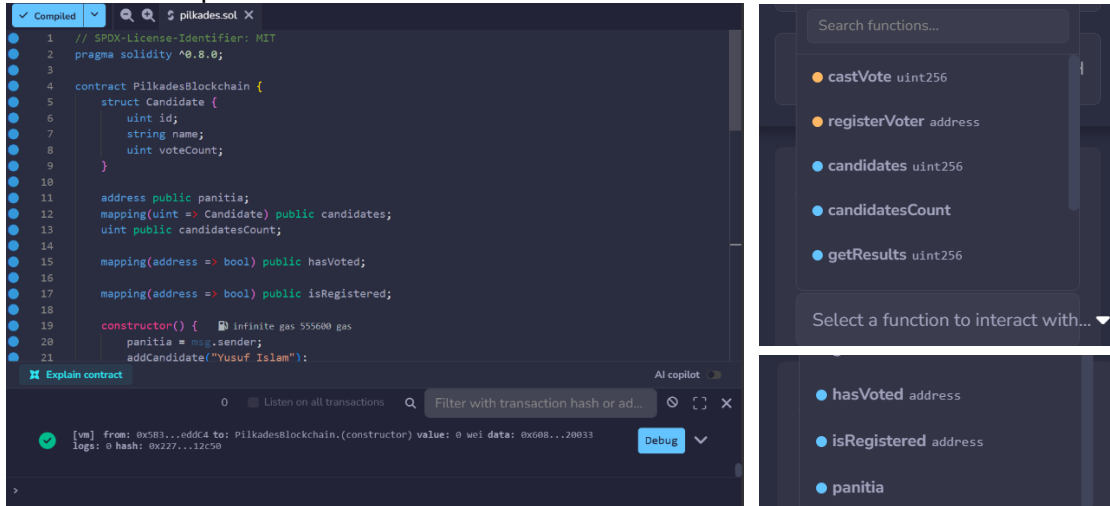
Meskipun demikian, sistem masih memiliki beberapa keterbatasan, seperti ketergantungan pada Firebase untuk autentikasi, proses registrasi pemilih yang masih dilakukan secara manual, serta penggunaan MetaMask sebagai satu-satunya wallet yang didukung. Oleh karena itu, pengembangan lebih lanjut diperlukan agar sistem memiliki tingkat keamanan, efisiensi, dan kemudahan penggunaan yang lebih baik sehingga siap diterapkan pada lingkungan yang lebih luas.

DAFTAR PUSTAKA

- Ahn, B. (2022). Implementation and Early Adoption of an Ethereum-Based Electronic Voting System for the Prevention of Fraudulent Voting. *Sustainability (Switzerland)*, 14(5). <https://doi.org/10.3390/su14052917>
- Alim, M. N. F., Azlan, F., & Miru, A. I. (2025). Perancangan dan Implementasi Sistem E-Voting Menggunakan. 10(2), 159–168. <https://doi.org/10.55679/semantik.v11i1.138>
- Guo, H., Xu, M., Zhang, J., Liu, C., Ranjan, R., Yu, D., & Cheng, X. (2024). BFT-DSN: A Byzantine Fault-Tolerant Decentralized Storage Network. *IEEE Transactions on Computers*, 73(5), 1300–1312. <https://doi.org/10.1109/TC.2024.3365953>
- Razali, M. H., Jamal, A. A., Fadzli, S. A., Zakaria, M. D., Wan Nik, W. N. S., & Hassan, H. (2025). e-Voting on Ethereum Blockchain. *Journal of Advanced Research in Applied Sciences and Engineering Technology*, 50(2), 186–194. <https://doi.org/10.37934/araset.50.2.186194>
- Saroinson, G. L., Sambul, A., & Sompie, S. R. U. A. (2025). E-Voting Systems. 20(1), 11–18.

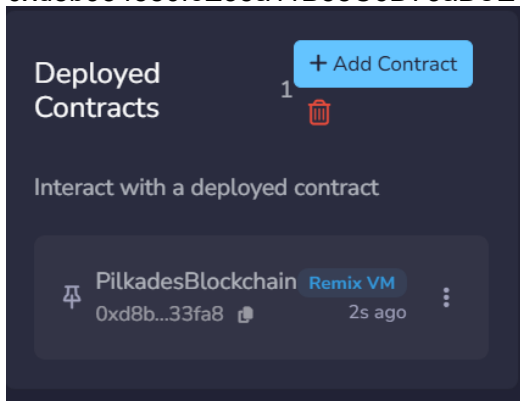
LAMPIRAN

1. Screenshot Cuplikan Kode & Function:



2. Contract Address:

0xd8b934580fcE35a11B58C6D73aDeE468a2833fa8



3. Screenshot Skenario Pengujian

Registrasi Pemilih Baru

Masukan Alamat Wallet (0x....)

Daftarkan Pemilih

✓ Berhasil Didaftarkan!

Riwayat Pendaftaran

0xAbbA6C47...6109	✓ Successful
0x702f7E1A...A421	✓ Successful

Registrasi Pemilih Baru

0xAbbA6C4792d2c5CaBd95Dc726882834787296109

Daftarkan Pemilih

✗ Gagal: Pastikan Anda Akun Panitia!

Riwayat Pendaftaran

0x3682365b...0019	✓ Successful
0x702f7E1A...A421	✓ Successful

Nomor Urut 01:
Yusuf Islam

Membangun Desa Bersama

☒

Nomor Urut 02:
Raehan Rahmat Zaki

Membangunkan Orang

☐

Nomor Urut 03:
Ade Nafila

19 A Lapangan Pekerjaan

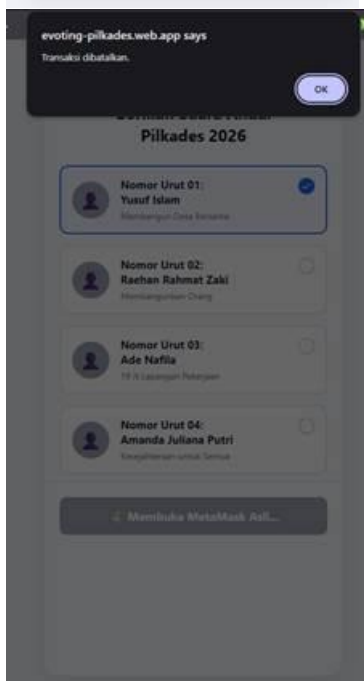
☐

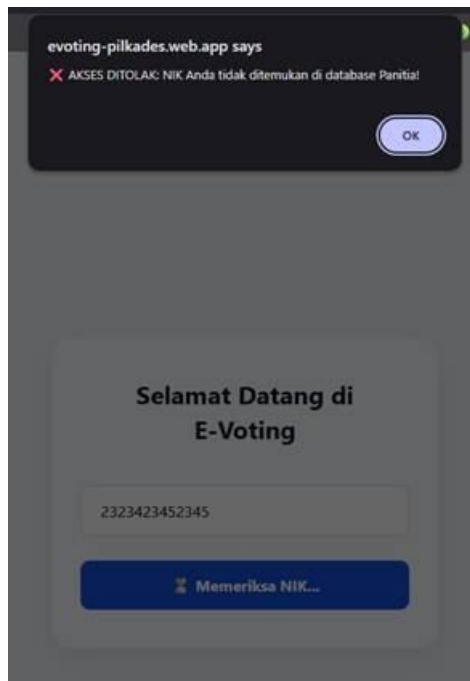
Nomor Urut 04:
Amanda Juliana Putri

Kesejahteraan untuk Semua

☐

Kirim Suara





4. Link Repository:
https://github.com/hanns1/blockchain-evoting-pilkades_kelompok-6